

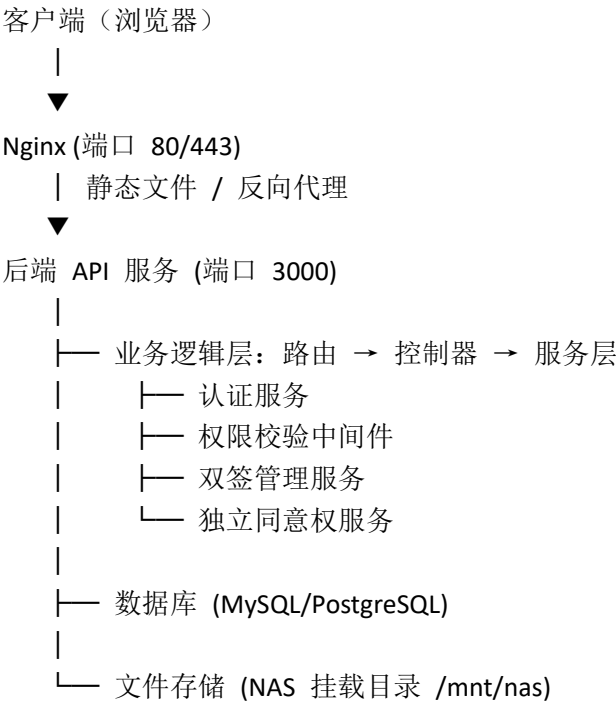
以下是协会管理系统的**后端蓝图**，描述了服务端架构、API 设计、数据模型、业务逻辑流程等，与前端蓝图对应，可直接作为后端开发的参考。

一、技术选型建议

- **语言/框架**：Node.js + Express（轻量快速），或 Python + Flask/FastAPI，或 Java + Spring Boot。根据你熟悉程度选择。
- **数据库**：MySQL 或 PostgreSQL（关系型，适合审批、关联数据）。
- **文件存储**：NAS 挂载目录（通过 NFS/SMB 挂载到服务器本地路径，如 `/mnt/nas_attachments``）。
- **认证**：简单的 Session/Cookie 或 JWT Token，因账号固定，无需 OAuth。
- **部署**：物理服务器上运行后端进程，由 Nginx 反向代理。

二、总体架构

...



...

- 所有前端请求发往 `/api/...`，由 Nginx 代理到后端。
- 文件上传：后端接收文件，保存到 NAS 挂载路径，数据库只存储相对路径或文件名。

三、数据库表设计（核心表）

1. accounts（账号表，固定数据）

- id, username, password（哈希存储）, role（admin/vice1/vice2/vice3/minister/academic）, real_name

2. members（成员）

- id, name, student_id, phone, role_tag（成员/干事）, created_at, updated_at
- **待变更**时，另存为 pending_changes 表

3. cadre_records（干事档案）

- id, member_id, activity_date, activity_name, description, created_at, updated_at

4. resource_requests（人力物力申请）

- id, applicant_id（部长账号 id）, type（物资/场地）, item_name, quantity, purpose, use_date, status（枚举：待组织副会审、待会长同意、已批准、已驳回）, org_vice_opinion, president_opinion, created_at, updated_at

5. activities（活动记录）

- id, name, date, location, plan_text（或 plan_file_path）, second_class_desc, participant_ids（JSON 数组或关联表）, photo_links（JSON 数组）, created_at, updated_at

6. project_applications（项目申请）

- id, applicant_id, project_name, leader, member_ids（JSON）, budget, start_date, end_date, description, status（待学术副会审、待会长同意、已确认、已驳回）, acad_vice_opinion, president_opinion, created_at

7. academic_materials（学术资料）

- id, title, summary, file_path, created_at, updated_at

8. pending_changes（双签变更表，通用）

- id, change_type（成员新增/修改/删除、干事档案新增/修改/删除、活动增删改、学术资料增删改等）, module, action, proposed_data（JSON，存储拟变更的内容）, original_data（JSON，用于对比）, proposer_id, confirm_status（待会长确认/待副会确认）, confirmer_id, used_special_power（布尔，是否使用了独立同意权）, created_at, updated_at

9. president_power（会长独立同意权记录）

- id, president_id, period_start（半年周期起始日期）, total_used（已用次数）, max_uses（固定 3）

四、API 路由设计（按资源模块）

所有 API 以 `/api` 为前缀，需验证登录（除登录接口外）。

1. 认证

- `POST /api/login`

请求: `{ username, password }`

响应: `{ token, role, userInfo }`

登录后生成 JWT 或 session。

2. 成员资料（需权限）

- `GET /api/members` —— 列表（查询参数: search, role_tag）

- `GET /api/members/:id` —— 详情

- `POST /api/members` —— 新增（行政副会/会长发起，进入待确认）

- `PUT /api/members/:id` —— 修改（同上）

- `DELETE /api/members/:id` —— 删除（同上）

- `POST /api/members/batch` —— 批量登记（需行政副会/会长权限，直接生效？或者也走双签？可根据需求决定，建议批量也走双签流程，生成一条批量变更记录）

3. 干事档案

- `GET /api/cadre-records` —— 列表

- `POST /api/cadre-records` —— 新增（待确认）

- `PUT /api/cadre-records/:id` —— 修改（待确认）

- `DELETE /api/cadre-records/:id` —— 删除（待确认）

4. 人力物力申请

- `GET /api/resource-requests` —— 列表（按角色过滤：部长看自己的，组织副会看全部，会长看全部）

- `POST /api/resource-requests` —— 提交申请（部长）

- `PUT /api/resource-requests/:id/approve-org` —— 组织副会审批（请求: `{ action: 'agree'/'reject', comment }`）

- `PUT /api/resource-requests/:id/approve-president` —— 会长审批（action + 是否使用独立同意权布尔标志）

5. 活动记录

- `GET /api/activities` —— 列表

- `POST /api/activities` —— 新增（待确认）

- `PUT /api/activities/:id` —— 修改（待确认）

- `DELETE /api/activities/:id` —— 删除（待确认）

6. 项目申请

- `GET /api/projects` —— 列表

- `POST /api/projects` —— 提交（学术部长）

- `PUT /api/projects/:id/approve-acad` —— 学术副会审批

- `PUT /api/projects/:id/approve-president` —— 会长审批

7. 学术资料

- `GET /api/materials` —— 列表
- `POST /api/materials` —— 新增（待确认）
- `PUT /api/materials/:id` —— 修改（待确认）
- `DELETE /api/materials/:id` —— 删除（待确认）

8. 待办与确认（双签核心）

- `GET /api/pending-tasks` —— 获取当前用户需要处理的所有待确认变更和待审批申请（数量用于红点）
- `GET /api/pending-changes` —— 列出待确认的变更列表（只显示该用户需确认的）
- `PUT /api/pending-changes/:id/confirm` —— 确认或驳回变更（请求：{ action: 'confirm'/'reject', reason }）
 - 如果会长操作且使用独立同意权，传递 `use\_power: true`，此时跳过副会确认，直接应用变更，并扣减次数。

9. 会长特权

- `GET /api/president/power-status` —— 获取剩余次数和有效期
- `PUT /api/resource-requests/:id/approve-president` 和 `PUT /api/projects/:id/approve-president` 及 `PUT /api/pending-changes/:id/confirm` 中可以附带 `use\_power: true`

10. 文件上传

- `POST /api/upload` —— 通用文件上传接口，返回文件路径/URL，前端在提交表单时带上附件信息。或者直接集成在资源创建接口中（multipart/form-data）。

五、业务逻辑流程（关键）

1. 双签变更流程（成员、档案、活动、学术资料）

1. 发起者（如行政副会）提交新增/修改/删除请求 → 后端创建 pending_change，状态为“待会长确认”，不修改实际数据表。
2. 会长登录看到待办 → 点击确认 → 后端根据 change_type 执行真实数据库操作，将 pending_change 状态标记为已完成。
 - 若会长驳回 → 变更丢弃（或标记已驳回），通知发起者。
3. 如果发起者是会长，则状态为“待对应副会确认”，流程对称。
4. 会长使用独立同意权：在确认时直接跳过副会确认，应用变更，并记录 used_special_power=true，扣减次数。

2. 申请审批流程（人力物力、项目）

1. 申请人提交 → 创建记录，状态“待副会审”。
2. 分管副会操作同意 → 状态变为“待会长同意”；若驳回 → 状态“已驳回”，结束。

- 3. 会长同意 → 状态“已批准”/“已确认”；驳回 → “已驳回”。
- 4. 会长独立同意权：可在“待副会审”或“待会长同意”状态下直接批准，跳过副会步骤，消耗次数。

3. 会长独立同意权管理

- 数据库 `president_power` 表记录每个半年周期的使用情况。每次使用检查当前周期：如当前时间在 ``period_start`` 对应的半年内，且 `used < 3`，则允许并增加 `used`。否则拒绝。
- 周期边界由后端判断（1月1日/7月1日），新周期自动插入新记录或重置计数。

六、中间件设计

- ****认证中间件****：验证 `token/session`，将用户信息（`id, role`）挂载到请求上下文。
- ****角色权限中间件****：配置每个路由需要的角色（例如 ``requireRole('admin', 'vice1')``），在路由处理前检查。
- ****待办计数器****：可注入到响应头或单独接口，方便前端红点提示。

七、文件处理逻辑

- 上传时：检查文件扩展名（`docx/xlsx/pdf`），大小限制（`2MB`），通过后转存到 ``/mnt/nas/attachments/`` 下按日期分类的目录，生成唯一文件名。
- 数据库存储文件路径，返回给前端下载 URL（如 ``/api/files/:filename``）。
- 下载接口：``GET /api/files/:filename``，后端从 NAS 读取文件流并返回，设置正确的 `Content-Type` 和 `Content-Disposition` 头以触发下载。

八、安全与备份

- 密码使用 `bcrypt` 哈希存储。
- 所有 API 除登录外需认证。
- 数据库每日自动备份脚本（`crontab`）将 `dump` 保存到 NAS 备份目录。
- 日志记录关键操作（审批、变更、使用特权），便于追溯。

这份后端蓝图与前端蓝图完全匹配，角色权限、双签逻辑、特权使用、文件存储等均有明确设计，可直接指导后端技术实现。